```
  def price_in_cents=(cents)
    @price = cents / 100.0
  end
  # ...
end

book = BookInStock.new("isbn1", 33.80)
puts "Price          = #{book.price}"
puts "Price in cents = #{book.price_in_cents}"
book.price_in_cents = 1234
puts "Price          = #{book.price}"
puts "Price in cents = #{book.price_in_cents}"
```

produces:

```
Price          = 33.8
Price in cents = 3380
Price          = 12.34
Price in cents = 1234
```

Here we've used attribute methods to create a virtual instance variable. To the outside world, price_in_cents seems to be an attribute like any other. Internally, though, it has no corresponding instance variable.

This is more than a curiosity. In his landmark book, *Object-Oriented Software Construction,* Bertrand Meyer calls this the *Uniform Access Principle*. By hiding the difference between instance variables and calculated values, you are shielding the rest of the world from the implementation of your class. You're free to change how things work in the future without impacting the millions of lines of code that use your class—for example, you could switch from a float to a BigDecimal and the users of this class would never need to know. This is a big win.

Attributes, Instance Variables, and Methods

This description of attributes may leave you thinking that they're nothing more than methods—why'd we need to invent a fancy name for them? In a way, that's absolutely right. An attribute *is* just a method. Sometimes an attribute simply returns the value of an instance variable. Sometimes an attribute returns the result of a calculation. And sometimes those funky methods with equals signs at the end of their names are used to update the state of an object. So, the question is, where do attributes stop and regular methods begin? What makes something an attribute and not just a plain old method? Ultimately, that's one of those "angels on a pinhead" questions. Here's a personal take.

When you design a class, you decide what internal state it has and also decide how that state is to appear on the outside to users of your class. The internal state is held in instance variables. The external state is exposed through methods we're calling *attributes*. And the other actions your class can perform are just regular methods. It really isn't a crucially important distinction, but by calling the external state of an object its attributes, you're helping clue people in to how they should view the class you've written.

Classes Working with Other Classes

Our original challenge was to read in data from multiple CSV files and produce various simple reports. So far, all we have is BookInStock, a class that represents the data for one book.